Glass Ballroom Verification: Client State Attestation in Untrusted Environments

Dante S. Ielceanu
Los Gatos High School
dante.ielceanu@gmail.com

Abstract

Modern web systems rely heavily on client-side execution to render and transform application state, yet servers lack visibility into how transmitted data is ultimately presented or manipulated within untrusted client environments. While cryptographic transport guarantees the integrity of server-to-client communication, it provides no assurance over the correctness of rendered state following client-side hydration, scripting, and dynamic modification. This blind spot enables a wide class of state forgery, automation abuse, and falsified attestations.

We present **Glass Ballroom Verification (GBV)**, a deterministic client state attestation pipeline that verifies rendered application state without requiring platform-side APIs, trusted execution environments, or hardware support. GBV operates by issuing a temporal challenge and collecting synchronized observations across multiple independent client surfaces. These observations are reconciled through semantic invariants and deterministic canonicalization, producing a cryptographic commitment over the coherent core state. The resulting signed receipt enables third-party verification with minimal disclosure through Merkle proofs.

Rather than attempting to eliminate forgery entirely, GBV raises the economic and technical cost of producing coherent falsified state across heterogeneous surfaces within bounded time. We describe the GBV protocol, its threat model, and a reference implementation. Open-source development of GBV is currently in progress.

Keywords: Client State Attestation; Untrusted Environments; Rendered State Verification; Multi-Surface Coherence; Canonicalization; Merkle Commitments; Adversarial Systems

Introduction

In a grand glass ballroom, two partners perform an elegant dance. Their movements are precise, coordinated, and governed by a strict routine. Around the ballroom stand observers positioned at different angles, each viewing the dance through transparent walls. From one side, a step may appear perfectly executed. From another, the same movement may reveal a subtle misalignment. The performance is considered valid only if the dance appears coherent from all perspectives.

The dancers trust that no single observer defines the truth of the routine. Instead, correctness emerges from consistency across independent views. The question is not whether imperfections can exist, but how much deviation can occur before the dance no longer satisfies the choreography.

Modern web systems operate within a similar glass ballroom. A server transmits data to a client, but the client environment is responsible for rendering, transforming, and presenting that state. Scripts execute, layouts shift, content hydrates, and dynamic logic reshapes what users ultimately see. Yet the server observes only what it sends, not what the client ultimately produces.

This disconnect creates a fundamental security gap: servers can cryptographically guarantee what they transmit, but cannot verify what is actually rendered or manipulated within untrusted client environments. Adversaries may alter DOM state, intercept and modify runtime behavior, automate falsified interfaces, or simulate entire application environments while still receiving cryptographically valid server responses.

We refer to this challenge as the *glass ballroom problem*: ensuring that client-visible application state remains coherent across independent observational perspectives, despite full adversarial control of the client environment.

The Limits of Transport-Level Trust

Existing security mechanisms largely terminate trust at the network boundary. TLS ensures confidentiality and integrity of transmitted data, while server-side signing can attest to payload authenticity. However, these guarantees stop once data reaches the client runtime.

Single-page applications routinely transform raw payloads through hydration, rendering frameworks, and dynamic scripting. Business logic frequently executes exclusively on the client, and UI state is shaped by both server input and local computation. As a result, the state experienced by users may diverge substantially from the data transmitted by the server.

An attacker may therefore manipulate rendered output without violating transport security. Common techniques include DOM injection, runtime patching, automation frameworks, local TLS interception, replay of stale state, and full environment simulation. From the server's perspective, all cryptographic checks may pass while the actual application state has been falsified.

Why Verifying Rendered State Is Hard

Attesting client-visible state presents several challenges:

- The client environment is fully adversarial and programmable.
- Rendered state evolves dynamically through scripts and frameworks.

- Minor UI drift occurs naturally due to personalization, experiments, and layout variability.
- Naive evidence collection is easily spoofed.

Hardware-based trusted execution environments offer potential isolation but are impractical for widespread browser deployment. Platform-side APIs for state verification require cooperation from application providers and cannot be assumed in open environments.

Overview of GBV

GBV addresses the glass ballroom problem by enforcing coherence across independent client surfaces within bounded time.

Rather than trusting a single rendered view, GBV issues a temporal challenge that triggers synchronized observation of application state from multiple heterogeneous client surfaces. These surfaces expose overlapping semantic facts about the underlying state.

The pipeline proceeds as follows:

1. A verifier issues a nonce-bound temporal challenge specifying a surface observation plan and semantic constraints.
2. The prover collects rendered state snapshots across multiple independent surfaces.
3. Semantic invariants are extracted and reconciled to identify coherent core facts.
4. The reconciled state is deterministically canonicalized to tolerate benign drift.
5. A cryptographic commitment is constructed and signed, producing a verifiable attestation receipt.

Contributions

This paper makes the following contributions:

- Formalization of the glass ballroom problem for client-rendered state verification.
- A deterministic multi-surface attestation pipeline for hostile client environments.
- A semantic coherence mechanism enforcing cross-surface invariants.
- A canonicalization strategy resilient to UI drift.
- A reference implementation with ongoing open-source development.

Threat Model and Security Objectives

GBV operates in hostile client environments where the prover controls local execution. Adversaries may inspect, modify, intercept, automate, or simulate all client-side behavior, including browser state, runtime logic, and network interactions.

We do not assume the presence of trusted hardware, secure enclaves, or cooperative platform APIs. The verifier observes only network-level communication and the outputs of the GBV pipeline.

Adversarial Capabilities

We classify adversaries into three escalating tiers:

1. **Manual manipulation:** Direct modification of rendered state using developer tools.
2. **Automated forgery:** Scripted injection, masking, and dynamic DOM rewriting synchronized with challenges.
3. **Simulated environments:** Fully virtualized clients presenting coherent but fraudulent application state across surfaces.

All tiers are assumed capable of intercepting and modifying local network traffic, replaying stale state, and responding programmatically to verifier challenges.

Security Goal

GBV does not attempt to make forgery theoretically impossible. Instead, it enforces *economic security*: increasing the technical complexity and operational cost of sustaining coherent falsified state beyond rational utility.

Formally, for adversary $\mathcal{A}$:

$$\text{Cost}(\mathcal{A}_{\text{forgery}}) > \text{Utility}(\mathcal{A}_{\text{fraud}}) \tag{1}$$

When this inequality holds, forgery becomes irrational under realistic constraints.

Threats Outside Scope

GBV does not defend against:

- Kernel-level compromise of the host operating system
- Hardware emulation indistinguishable from genuine platforms
- Physical capture attacks with unrestricted forensic access

These threats require trusted hardware roots and lie outside browser-based attestation scope.

System Overview

GBV transforms hostile client sessions into cryptographically verifiable observations through a deterministic multi-surface pipeline.

Let σ denote the rendered client-visible application state at time t. Direct verification of σ is infeasible due to adversarial control and non-deterministic noise. Instead, GBV extracts overlapping semantic representations of σ across k independent surfaces.

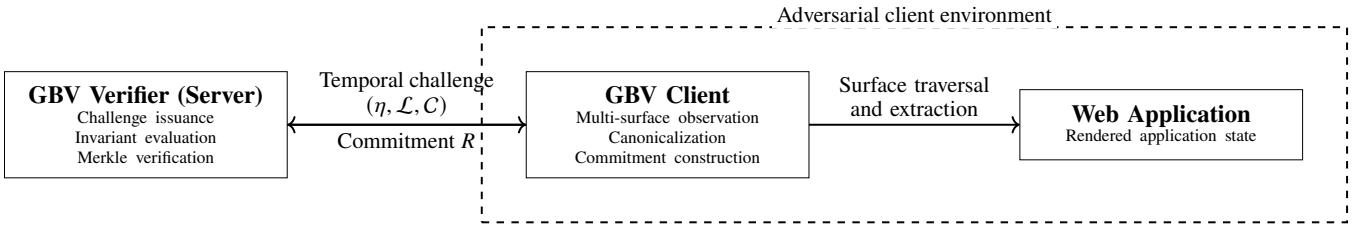

Figure 1: GBV system architecture and trust boundary. The verifier is trusted, while all components within the dashed region operate in an adversarial client environment.

Each surface exposes partial but correlated views of the underlying state, as illustrated in Figure 1.

Core Intuition

Verification based on a single rendered view is brittle: any one surface can be forged in isolation through DOM manipulation, script injection, or environment simulation.

GBV instead relies on *cross-surface coherence*. While individual representations may be easy to falsify, maintaining semantic consistency across heterogeneous surfaces within a bounded time window requires coordinated manipulation of multiple subsystems.

As surface diversity increases, adversaries must synchronize changes across distinct rendering paths, metadata sources, and transformation logic. Deviations that remain invisible from one perspective become detectable when viewed from others.

This mirrors the glass ballroom metaphor: correctness emerges not from any single observation, but from consistency across independent views.

Pipeline Stages

GBV operates in five deterministic stages:

1. **Temporal Challenge Issuance:** The verifier issues an ephemeral nonce η and surface traversal plan $\mathcal{L}$.
2. **Multi-Surface Observation:** The prover collects synchronized rendered snapshots embedding η.
3. **Semantic Coherence Enforcement:** Extracted facts are reconciled against invariant constraints C.
4. **Deterministic Canonicalization:** Noisy representations are reduced to a stable core state.
5. **Cryptographic Commitment:** The canonical state is committed via Merkle hashing and signed.

Each stage is deterministic: for a fixed surface plan, invariant set, and rendered state, GBV produces identical outcomes across repeated executions.

Why Multi-Surface Coherence Is Hard to Forge

Each observation surface typically draws from distinct application subsystems, such as:

- Public UI render paths
- Internal telemetry attributes
- Shadow metadata
- Alternate navigation views
- Server-derived and client-derived transformations

While a single DOM injection can alter visible UI, maintaining coherent shadow facts, telemetry metadata, and alternate representations simultaneously requires replicating substantial portions of application logic.

Temporal nonces further prevent replay and stale-state reuse by forcing adversaries to respond in real time.

As surface diversity increases, the engineering effort required to sustain coherent forgery grows rapidly.

Design Principles

GBV follows four guiding principles:

- **No trusted client assumptions**
- **No platform-side cooperation required**
- **Tolerance to benign UI drift**
- **Escalating adversarial cost**

These principles prioritize deployability while ensuring that adversarial effort scales faster than honest execution cost.

Hybrid Evidence Model

GBV does not assume uniform semantic accessibility across providers or application surfaces. Some platforms expose stable semantic artifacts such as certificate identifiers, course slugs, or embedded metadata, while others expose only presentation-layer evidence derived from rendered DOM content.

To accommodate this variability, GBV adopts a *hybrid evidence model*. Each observation surface contributes a mixed-signal evidence bundle composed of both semantic artifacts (when available) and presentation-derived facts. Verification decisions are based on cross-surface consistency over this combined evidence set rather than reliance on any single signal.

Presentation-layer evidence is never trusted in isolation. It is only accepted when corroborated by structural invariants or reinforced by agreement across independent surfaces. This design allows GBV to remain robust across heterogeneous platforms without introducing false negatives due to missing, stale, or inconsistently exposed semantic identifiers.

Unless otherwise specified, experiments enforce structural and nonce-bound invariants but do not require strict agreement between cosmetic identifiers (e.g., rendered titles) and semantic identifiers (e.g., internal slugs). Some providers legitimately expose divergent identifiers across views; enforcing strict coupling would degrade liveness without materially strengthening attestation security.

Formal State Model

We model the client-visible rendered application state at time t as a directed acyclic graph over the Document Object Model (DOM):

$$\sigma_t = (V_t, E_t)$$

where vertices represent DOM nodes and edges represent structural containment and rendering relationships.

Rendered state is inherently noisy. Layout shifts, experiment variants, personalization, timestamps, advertisements, and session artifacts introduce variation unrelated to semantic correctness.

Direct comparison of rendered state across executions is therefore brittle. GBV instead targets a stable semantic abstraction of rendered state that tolerates benign drift while preserving meaningful application facts.

Semantic Core

To isolate meaningful content, we define a *semantic core subgraph*:

$$\sigma_s \subset \sigma_t$$

which captures invariant application facts relevant to attestation.

The semantic core excludes layout-only elements, volatile attributes, and session-specific artifacts, while preserving identifiers, progress markers, status flags, and other semantically meaningful values.

GBV does not require the semantic core to be identical across surfaces; instead, it assumes that critical semantic values appear redundantly across multiple independent views.

Observation Mapping

Each surface observation applies an extraction function:

$$\Phi_i : \sigma_t \to F_i$$

where F_i is a finite set of structured facts extracted from surface i.

Facts may include identifiers, progress markers, status values, embedded metadata, or shadow attributes. Different surfaces may expose overlapping facts through distinct representations.

The role of GBV is not to enforce identical representations, but to reconcile these fact sets through invariant constraints that encode semantic consistency.

Deterministic Canonicalization

Because σ_t varies naturally across time, clients, and executions, GBV applies a deterministic canonicalization function:

$$f_{can} : \sigma_t \to L$$

that maps noisy rendered state into a stable leaf set L representing the semantic core.

Canonicalization removes sources of benign variability, including:

- Layout-only DOM elements
- Session-specific tokens
- Volatile attributes and timestamps
- Presentation-only formatting

while preserving semantically meaningful content such as identifiers, progress markers, state flags, and embedded metadata.

The canonicalizer is designed to be deterministic over the challenge window Δt:

$$\forall t_1, t_2 \in \Delta t : \; f_{can}(\sigma_{t_1}) = f_{can}(\sigma_{t_2}) \tag{2}$$

This ensures that benign UI drift does not affect attestation outcomes, while adversarial manipulation that alters semantic content remains detectable.

Surface Independence

Each observation surface exposes overlapping but non-identical subsets of semantic facts.

Let:

$$F_i = \{f_{i1}, f_{i2}, \ldots, f_{im}\}$$

denote the fact set extracted from surface i.

GBV does not assume that any single surface is authoritative. Instead, it assumes that for a coherent honest state, critical semantic values appear redundantly across multiple independent surfaces:

$$\bigcap_{i=1}^{k} F_i \neq \emptyset$$

Surface independence arises naturally in modern applications, where different views draw from distinct rendering paths, data sources, and transformation pipelines. While a single surface may be trivially forged, maintaining consistency across heterogeneous surfaces requires synchronized manipulation of multiple subsystems.

This redundancy forms the structural basis for adversarial detection.

Semantic Invariants

GBV enforces coherence by evaluating invariant constraints over extracted fact sets.

Let C denote a collection of invariant relations applied across surfaces. Invariants encode semantic expectations that must hold for a coherent application state.

Typical invariants enforce equality, ordering, or bounded deviation between corresponding facts:

$$\forall j \in \text{CoreFacts} : \ \delta(F_1[j], F_2[j], \ldots, F_k[j]) \leq \epsilon \tag{3}$$

where δ is a domain-specific distance function and ϵ tolerates benign variation.

Violations of any invariant result in immediate rejection.

Examples of Invariants

Examples of enforced invariants include:

- Progress percentage consistency across UI and telemetry metadata
- Identifier coherence across public and internal representations
- Timestamp ordering constraints across views
- State flags matching across surfaces

These invariants are selected to be stable under honest execution while difficult to maintain under adversarial manipulation.

Why Invariants Resist Simple Forgery

Naive forgery techniques typically modify visible UI elements in isolation. However, shadow attributes, telemetry-bound metadata, and alternate navigation views often derive from distinct runtime data flows.

Maintaining invariant satisfaction across these flows requires coordinated manipulation of multiple subsystems in real time. This substantially increases adversarial engineering effort and reduces the feasibility of simple script-based attacks.

Semantic invariants therefore serve as the enforcement mechanism that transforms surface redundancy into concrete security guarantees.

GBV Protocol Specification

GBV converts hostile client observations into verifiable attestations through a deterministic five-stage pipeline. Each stage is fully specified, non-interactive after challenge issuance, and produces no observable side channels beyond the final acceptance or rejection outcome.

Stage I: Temporal Challenge Issuance

The verifier $\mathcal{V}$ issues an ephemeral challenge consisting of:

- A nonce η
- A surface traversal plan $\mathcal{L} = \{S_1, S_2, \ldots, S_k\}$
- A semantic constraint set C

Formally:

$$\mathcal{V} \xrightarrow{\eta, \mathcal{L}, C} \mathcal{P} \tag{4}$$

The nonce is bound to a short validity window Δt and uniquely identifies the challenge instance. This prevents replay, precomputation, and stale-state reuse.

The traversal plan specifies which surfaces must be observed and in what order, but does not expose verifier-side expectations about extracted values.

Stage II: Multi-Surface Observation

Upon receiving the challenge, the prover traverses each surface $S_i \in \mathcal{L}$ within the validity window Δt.

For each surface, the prover captures a rendered snapshot and injects the nonce into the DOM prior to serialization. Each observation produces:

$$\sigma_i = \{\text{DOM}_i, \eta, \text{URL}_i\} \tag{5}$$

Nonce injection cryptographically binds observations to the challenge instance and prevents replay of pre-rendered or cached state.

Optional local timestamps may be recorded for debugging and performance analysis, but freshness is enforced solely via the nonce and verifier-side validation.

Stage III: Semantic Coherence Enforcement

From each observed state σ_i, structured semantic facts F_i are extracted using deterministic extraction rules.

The verifier evaluates all extracted facts against the invariant constraint set C:

$$\forall j \in C : \ \text{Check}_j(F_1, F_2, \ldots, F_k) = \text{true} \tag{6}$$

If any invariant check fails, verification terminates immediately and the attestation is rejected.

Only fact sets satisfying all constraints are considered semantically coherent and eligible for canonicalization.

Stage IV: Deterministic Canonicalization

Each accepted surface snapshot is processed by the canonicalization function:

$$L_i = f_{can}(\sigma_i)$$

Canonicalization removes benign variability while preserving semantic content. The final canonical leaf set is defined as:

$$L = \bigcup_{i=1}^{k} L_i$$

Duplicate semantic elements are merged deterministically. The resulting set L represents the coherent semantic core of the rendered application state.

Because canonicalization is deterministic, identical coherent states always produce identical leaf sets, regardless of traversal timing or surface order.

Stage V: Cryptographic Finality

Leaves in L are hashed into a Merkle tree using a fixed ordering.

Let R denote the Merkle root. The verifier issues the signed attestation:

$$\mathcal{A}_{tt} = \text{Sign}_{\mathcal{V}}(R, \eta, \text{SubjectID}) \tag{7}$$

The attestation binds the canonicalized state to the challenge nonce and subject identity.

Selective disclosure is supported via Merkle inclusion proofs, allowing third parties to verify individual semantic claims without access to raw DOM snapshots or unrelated state.

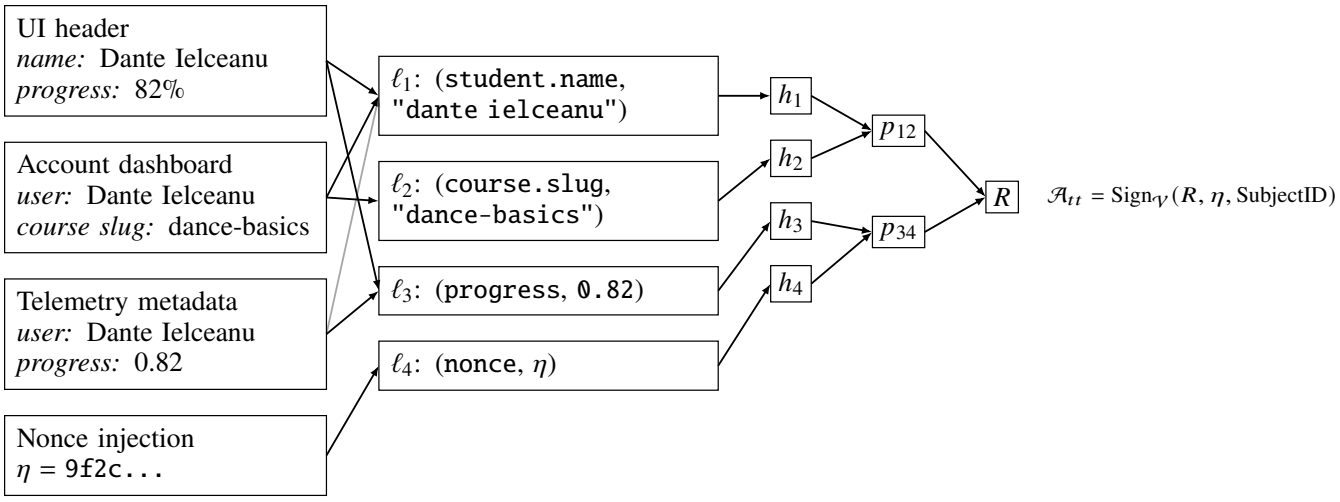

Figure 2: GBV semantic commitment flow. Multiple rendered surfaces expose overlapping representations of core application facts (e.g., student name "Dante Ielceanu"). These observations are deterministically canonicalized into semantic leaves L, which are committed via a Merkle root R for signing and selective disclosure.

Adversarial Resiliency Analysis

The central security property of GBV is that sustaining coherent falsified state across heterogeneous client surfaces within a bounded challenge window becomes increasingly costly as surface diversity increases.

Rather than attempting to prevent manipulation outright, GBV transforms forgery from a trivial single-surface operation into a synchronized multi-surface simulation problem.

Single-Surface Fragility

With a single observation surface ($k = 1$), forgery is trivial. An adversary may modify DOM elements, intercept network traffic, or inject scripts to present falsified rendered state while preserving cryptographic transport guarantees.

This limitation motivates the use of multiple independent surfaces: correctness cannot be inferred from any single rendered view.

Multi-Surface Simulation Attacks

A more capable adversary may attempt to simulate all k surfaces coherently.

To succeed, the adversary must:

- Detect temporal challenges in real time
- Generate consistent semantic facts across heterogeneous surfaces
- Replicate canonicalization-relevant transformations
- Preserve invariant satisfaction under UI drift and runtime variability

Let H_i denote the engineering complexity required to spoof surface i under canonicalization and invariant enforcement.

A coarse adversarial complexity proxy is:

$$C(\mathcal{A}) \propto k \cdot \sum_{i=1}^{k} H_i \tag{8}$$

As surfaces draw from distinct rendering paths and data flows, H_i increases with surface heterogeneity. The aggregate cost therefore grows superlinearly with k.

Parametric Forgery Scaling

Under a simplifying independence assumption, let p denote the probability that a single surface can be spoofed coherently within a challenge window. The probability of undetected forgery across k surfaces is

$$P_f = p^k \,. \tag{9}$$

This analysis is parametric (analytic) rather than empirical: in practice, spoofing events may be correlated due to shared data sources or attacker tooling. GBV therefore emphasizes surface diversity to reduce correlation and invalidate shared manipulation strategies.

Temporal Friction

The ephemeral challenge window Δt further increases adversarial difficulty.

Manual manipulation becomes infeasible as k increases, while automated systems must react in real time across multiple subsystems. This prevents precomputation and stale-state replay, forcing adversaries into fully online simulation.

Economic Security

The combined effects of:

- Surface diversity
- Deterministic canonicalization
- Semantic invariant enforcement
- Temporal unpredictability

produce escalating adversarial cost.

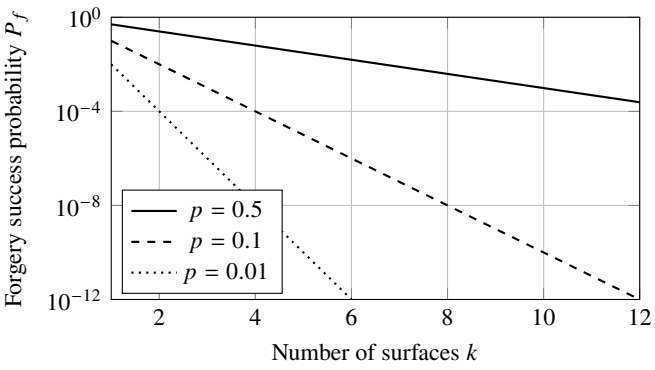

Figure 3: Analytic (parametric) forgery success scaling under an independence assumption, where $P_f = p^k$ decays exponentially with the number of independent surfaces k. The log-scale y axis highlights the separation between parameter regimes.

When the engineering effort required to sustain coherent falsified state exceeds the expected utility of fraud, rational adversaries are deterred. GBV therefore enforces economic security rather than absolute impossibility.

Implementation Case Study

To validate the practical feasibility of GBV, we developed a reference implementation targeting a large-scale single-page application platform. The implementation exercises all stages of the GBV pipeline, including temporal challenge issuance, multi-surface traversal, semantic invariant enforcement, deterministic canonicalization, and cryptographic commitment generation.

The system operates entirely within a browser extension context and does not rely on platform-side APIs, trusted execution environments, or hardware support.

An open-source implementation of GBV is under active development to support reproducibility, independent security analysis, and community-driven extension.

Surface Selection Criteria

Surface selection in GBV is not arbitrary. While the protocol itself is agnostic to which surfaces are chosen, security and liveness depend critically on selecting surfaces that expose overlapping semantic facts through heterogeneous rendering paths.

A verifier should select surfaces according to the following criteria:

- **Semantic overlap:** Each surface should expose at least one fact that participates in an invariant enforced across other surfaces (e.g., identifiers, progress markers, state flags).
- **Subsystem heterogeneity:** Surfaces should draw from distinct rendering pipelines, data sources, or transformation layers (e.g., UI views, telemetry-bound metadata, alternate navigation pages).
- **Manipulation asymmetry:** Forging one surface should not trivially imply the ability to forge others. Ideal surfaces require different manipulation techniques or tooling to spoof.
- **Traversal feasibility:** All selected surfaces must be reachable within the challenge window under typical network and rendering conditions.

Importantly, surface count alone is insufficient. Increasing k without increasing heterogeneity yields diminishing security returns. GBV security emerges from semantic intersection across independently derived views, not from raw multiplicity.

In practice, effective surface sets are constructed by identifying core semantic facts and selecting multiple representations of those facts that arise naturally across the application. This process can be performed manually during verifier configuration or iteratively refined based on observed invariant coverage and traversal reliability.

Surface Selection and Diversity

A primary design objective was selecting observation surfaces that draw from distinct rendering paths and data flows.

In the case study, selected surfaces included:

- Public progress and completion interfaces
- Alternate navigation views
- Telemetry-bound DOM attributes
- Certificate rendering pages
- Account dashboard summaries

While these surfaces expose overlapping semantic facts, they originate from different subsystems, rendering pipelines, and state representations. This heterogeneity is critical to increasing adversarial complexity under multi-surface verification.

Shadow Fact Extraction

Beyond visible UI elements, the implementation targets *shadow facts*: structured metadata embedded within DOM attributes for analytics and telemetry purposes.

These attributes frequently encode semantic values such as identifiers, progress markers, and state flags that are not directly visible to users but remain tightly coupled to application logic.

An illustrative extraction example is shown below:

```
const raw = el.getAttribute("data-click-value");
const obj = JSON.parse(raw);
const slug = obj?.product?.slug;
```

Shadow facts tend to remain stable across UI redesigns and are less likely to be modified by naive DOM-based forgery scripts, making them valuable anchors for invariant enforcement.

Traversal Plan

A traversal plan with $k = 6$ surfaces was selected to balance liveness and security.

The plan required navigating across multiple application subsystems within a bounded challenge window, including page transitions and asynchronous rendering delays.

Informal prototype testing demonstrated reliable traversal under typical network conditions, with overall runtime dominated by page loading rather than canonicalization or hashing overhead.

Invariant Enforcement

Core semantic values — such as internal identifiers, progress completion markers, and certificate metadata — were enforced across all selected surfaces.

Invariant violations resulted in immediate rejection, preventing partially coherent state from reaching the canonicalization and commitment stages.

Importantly, the implementation does not enforce strict agreement on cosmetic identifiers (e.g., rendered titles) unless corroborated by semantic or structural evidence. This avoids false rejections arising from benign UI drift or provider-specific inconsistencies.

Isolation Properties

Observation logic executes within an isolated browser extension world, preventing page scripts from directly intercepting or modifying GBV execution.

While this isolation does not provide absolute security against advanced attackers, it significantly raises the bar for simple script-based manipulation and reduces accidental interference from application code.

Together, these implementation choices demonstrate that GBV can be deployed in real-world browser environments without specialized platform support while maintaining the intended security and liveness properties.

Experimental Evaluation: Deterministic Multi-Surface Behavior

To characterize the practical behavior of GBV under both honest execution and minimal adversarial perturbation, we conducted a series of controlled prototype evaluations using the reference implementation described in the previous section.

The purpose of this evaluation is not to establish a universal security threshold, but to empirically validate three core properties of the system:

- Deterministic behavior under repeated execution
- Predictable scaling of runtime and canonicalized state size with surface count
- Emergent detection of semantic inconsistency once sufficient cross-surface redundancy is induced

All experiments were conducted using identical traversal logic, canonicalization rules, and invariant enforcement. Because GBV is deterministic by construction, repeated executions over the same surface set yield identical semantic outcomes.

Experimental Setup

Each experiment consisted of issuing a temporal challenge with surface count k, traversing the specified surfaces within the challenge window, extracting semantic facts, enforcing invariants, canonicalizing rendered state, and producing a signed attestation if verification succeeded.

We evaluated four surface counts: $k \in \{1, 2, 4, 6\}$. For each configuration, multiple runs were performed to confirm outcome stability and runtime consistency.

Two execution conditions were tested:

- **Honest execution:** The client rendered authentic application state without modification.
- **Tier-1 adversarial perturbation:** A single semantic identifier was altered on one surface while leaving all other surfaces unmodified.

The Tier-1 adversary represents a minimal but realistic forgery attempt, such as modifying a rendered identifier or shadow metadata value via DOM manipulation or scripted injection.

Honest Execution Results

Under honest execution, GBV accepted all runs across all evaluated surface counts. No false rejections were observed.

Runtime and canonicalized state size scaled monotonically with surface count, reflecting predictable traversal and extraction cost as additional surfaces were incorporated. Across repeated runs at fixed k, acceptance outcomes were invariant and runtime variance was minimal.

These results confirm that GBV introduces no stochastic behavior under honest execution and that system cost scales smoothly with surface count.

Table 1: GBV behavior under honest execution across surface counts

k	Acceptance Rate	Mean Runtime (ms)	Canonical Leaves
1	100%	~200	10
2	100%	~270	18
4	100%	~360	35
6	100%	~420	51

Tier-1 Adversarial Results

Under Tier-1 adversarial perturbation, acceptance behavior depended on the degree of semantic redundancy induced by the selected surface set.

For $k \in \{1, 2, 4\}$, the introduced inconsistency did not violate any enforced invariant. In these configurations, the altered identifier appeared on only a single surface and did not intersect with a sufficient number of invariant-enforced semantic facts to force contradiction.

For the evaluated $k = 6$ traversal plan, all Tier-1 runs failed semantic verification. In this configuration, the modified identifier conflicted with invariant-enforced facts appearing redundantly across multiple independent surfaces, resulting in deterministic rejection.

Because GBV is deterministic, repeated executions of the same perturbation at fixed k produced identical outcomes. Acceptance and rejection behavior was invariant across runs.

Interpretation

These results illustrate a central property of GBV: adversarial resistance is driven by *semantic intersection*, not surface count alone.

Increasing k does not inherently guarantee detection. Instead, detection emerges once the selected surface set induces sufficient overlap among invariant-enforced semantic facts. Below this point, localized inconsistencies may remain isolated. Once crossed, invariant enforcement forces global consistency, causing even minimal forgery to collapse into a detectable contradiction.

Importantly, this behavior is structural rather than probabilistic. For a fixed traversal plan and invariant set, acceptance outcomes are a deterministic function of the induced semantic overlap.

The observed rejection at $k = 6$ reflects properties of the evaluated surface selection, not a universal threshold. Alternative surface choices at the same k may or may not induce equivalent redundancy. Exploring surface selection strategies and higher-tier adversaries capable of coordinated multi-surface simulation is deferred to future work.

Together, these results establish baseline correctness, determinism, and first-order adversarial resistance under minimal but realistic perturbation.

Table 2: GBV behavior under Tier-1 adversarial perturbation across surface counts

k	Acceptance Rate	Verdict	Failure Mode
1	100%	Pass	–
2	100%	Pass	–
4	100%	Pass	–
6	0%	Fail	Semantic inconsistency

Engineering Trade-offs

GBV balances security, liveness, and deployability through a set of explicit engineering trade-offs. Rather than optimizing for a single axis, the design prioritizes practical deployability while ensuring adversarial cost scales meaningfully with system parameters.

Challenge Window Selection

The temporal challenge window Δt determines the time available for multi-surface traversal and observation.

A shorter window increases adversarial friction by limiting the time available for coordinated manipulation, but risks traversal failure due to network latency, asynchronous rendering, or rate limiting. A longer window improves liveness and reliability but provides additional time for adversarial response.

In the reference implementation, a challenge window of approximately $\Delta t \approx 60$ seconds provided a practical balance. This window reliably supported automated traversal across $k = 6$ surfaces under typical network conditions while rendering manual multi-surface manipulation infeasible.

Canonicalization Overhead

Canonicalization runtime scales approximately linearly with the number of extracted DOM nodes. In practice, overall runtime was dominated by page loading, rendering, and navigation latency rather than canonicalization or cryptographic operations.

Hashing and Merkle construction incurred negligible overhead relative to surface traversal costs. This suggests that GBV remains viable on resource-constrained client devices and that increasing surface diversity primarily impacts liveness rather than computation.

Isolated Execution Context

Observation logic executes within an isolated browser extension context.

This separation prevents page scripts from directly intercepting GBV extraction and canonicalization logic, preserving an observation gap between the adversarial environment and the attestation pipeline. While not providing absolute isolation, this design significantly complicates trivial script-based forgery and reduces the attack surface available to injected code.

Surface Diversity vs. Liveness

Increasing surface count k improves adversarial resistance but degrades traversal reliability due to cumulative latency, navigation complexity, and platform-imposed rate limits.

Empirically, diminishing security returns appeared beyond moderate surface counts. In informal testing, surface sets in the range $k \in [4, 7]$ provided meaningful redundancy without materially harming liveness on complex single-page applications.

This trade-off highlights that GBV security derives from *semantic diversity* rather than raw surface count. Carefully selecting surfaces that expose overlapping facts through heterogeneous subsystems is more effective than indiscriminately increasing k.

A comprehensive empirical study of this trade-off is deferred to future work.

Evaluation Methodology and Prototype Status

The design and analysis presented in this work are grounded in a fully functional reference implementation of the GBV pipeline. The prototype implements all protocol stages, including nonce issuance, multi-surface traversal, semantic invariant enforcement, deterministic canonicalization, and Merkle-based commitment generation.

The reference implementation described in this paper is available as an open-source prototype to support reproducibility and independent analysis; details are omitted here to preserve blind review.

At the time of writing, GBV has not undergone large-scale empirical benchmarking or controlled adversarial evaluation. No quantitative performance measurements, adversarial success rates, or longitudinal deployment studies are reported.

Accordingly, this work should be interpreted as:

- A protocol specification for client-rendered state attestation
- A systems design validated through end-to-end implementation
- An analytical security model grounded in adversarial reasoning

Correctness Validation

Correctness was established through repeated end-to-end prototype execution on real platforms. These executions verified that:

- Honest coherent states consistently produced valid attestations
- Injected inconsistencies across surfaces resulted in deterministic rejection
- Canonicalization tolerated benign UI drift within the challenge window

These checks validate that the protocol behaves as specified under both honest execution and minimal adversarial perturbation.

Security Reasoning

Security claims in this work are derived analytically from:

- The adversarial model defined in Section 2
- Enforced semantic invariants across heterogeneous surfaces
- The escalating complexity of synchronized multi-surface simulation
- Temporal challenge unpredictability

Probability models and complexity expressions are illustrative and intended to characterize scaling behavior rather than measured attack frequencies.

Liveness Observations

Traversal reliability and runtime behavior were observed informally during prototype operation.

While these observations are not statistically rigorous, they informed practical design choices such as surface count selection and challenge window duration. Formal benchmarking and stress testing under controlled conditions are deferred to future work.

Limitations, Threats, and Ethics

While GBV significantly raises the difficulty of client-side state forgery, it does not provide absolute guarantees. Several limitations remain inherent to browser-based attestation in hostile environments.

Local Proxy and Traffic Manipulation

A determined adversary may deploy TLS-terminating local proxies to intercept and modify network traffic prior to rendering.

Although GBV increases adversarial difficulty through temporal challenges and multi-surface coherence, sufficiently sophisticated traffic manipulation frameworks may still succeed. GBV does not attempt to defeat arbitrary local interception, but rather to make sustained coherent forgery economically and operationally costly.

Full Environment Simulation

Highly motivated attackers may attempt to simulate entire browser environments, including rendering engines, application logic, and telemetry behavior.

While GBV dramatically increases the engineering complexity required to maintain semantic coherence across heterogeneous surfaces, it does not theoretically preclude full-environment simulation attacks. Defenses against such attacks require trusted hardware roots or platform-level guarantees and lie outside the scope of this work.

Kernel-Level Compromise

GBV does not defend against compromised operating systems or hardware-level attacks.

An adversary with kernel-level control may arbitrarily manipulate execution, memory, and network behavior. Such

threats require hardware-backed trust anchors and are orthogonal to browser-based attestation mechanisms.

Privacy Considerations

GBV is designed to minimize data exposure.

Canonicalization emits only semantic facts required for attestation; raw DOM content need not be retained. Cryptographic commitments enable selective disclosure via Merkle proofs, allowing third parties to verify specific claims without accessing full client-rendered state.

Implementations should adhere to least-privilege extraction principles, minimize retention of rendered content, and avoid persistent storage of sensitive client-side data. Deployments should clearly document extracted invariants and disclosure policies to ensure informed user consent.

Future Work

Several directions remain for expanding and strengthening GBV beyond the scope of the present work.

Autonomous Canonicalization

Selector drift and interface redesign pose practical challenges for DOM-based surface extraction.

Future work may incorporate vision-language models as assistive mechanisms for semantic extraction from rendered content, enabling canonicalization that remains robust under substantial UI changes. Such approaches would complement, rather than replace, deterministic surface definitions, preserving GBV's core correctness guarantees while improving long-term deployability.

Empirical Adversarial Evaluation

While the current evaluation validates determinism and basic adversarial resistance, a more comprehensive empirical study remains an important direction for future work. Potential extensions include:

- Automated forgery attempts across increasing surface counts
- Performance benchmarking under varying network and execution conditions
- Correlation analysis between surface spoofing events

These measurements would further quantify practical security gains, adversarial cost scaling, and liveness trade-offs under realistic attack models.

Broader Application Domains

Although the reference implementation targets educational verification, the GBV framework is domain-agnostic. Potential application areas include:

- Digital credentials and certifications
- Financial and transactional workflows
- Compliance and audit reporting
- Client-side analytics integrity

Exploring domain-specific invariant selection and surface design remains an open area for future investigation.

Open-Source Development and Review

GBV is under active open-source development to facilitate independent verification, reproducibility, and external security review. Public repositories include protocol specifications, reference implementations, and testing frameworks, enabling community scrutiny and iterative refinement.

Conclusion

Modern web systems increasingly rely on client-side execution to render, transform, and present application state, yet servers remain blind to how that state ultimately materializes within untrusted environments. This gap enables a wide class of forgery, automation abuse, and falsified attestations that cannot be addressed by transport-level cryptography alone.

This paper introduced *Glass Ballroom Verification (GBV)*, a deterministic multi-surface client state attestation pipeline designed for hostile client environments. GBV enforces semantic coherence across heterogeneous rendered surfaces under ephemeral challenges, followed by deterministic canonicalization and cryptographic commitment. Rather than trusting any single view of client state, GBV derives correctness from consistency across independent perspectives.

The central insight of GBV is that while individual rendered surfaces are easy to manipulate in isolation, sustaining coherent falsified state across multiple independent surfaces within bounded time requires replicating substantial portions of application logic. By shifting verification from single-view inspection to multi-perspective coherence, GBV raises the economic and technical cost of forgery beyond rational adversarial utility.

A working reference implementation demonstrates that GBV is practical without requiring trusted hardware, platform-side APIs, or provider cooperation. Analytical reasoning and prototype evaluation show that adversarial resistance emerges structurally from semantic intersection, rather than probabilistically from surface count alone.

GBV does not claim to eliminate client-side forgery. Instead, it reframes the problem: from attempting to make manipulation impossible, to making coherent deception expensive, fragile, and difficult to sustain. In doing so, GBV opens a new design space for verifying client-rendered application state in modern web systems—one where trust is earned not through a single viewpoint, but through agreement across many.

Acknowledgements

The author thanks mentors, reviewers, and peers who provided feedback on early drafts of this work, as well as the open-source community whose tooling and documentation made rapid prototyping and experimentation possible. Any remaining errors or omissions are the author's own.

References

[1] Merkle, R. C. (1980). Protocols for public key cryptosystems. *Proceedings of the IEEE Symposium on Security and Privacy*.

[2] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519.

[3] Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.

[4] Zhang, F., et al. (2020). DECO: Liberating web data using decentralized oracles. *Proceedings of CCS '20*.

[5] Google Developers. (2026). Chrome Extension Manifest V3 Documentation.